

Static Security Research Report

MAS_AIO.cmd Windows and Office Activation Script

Prepared: 9 October 2026

Method: source-based static analysis; no execution performed

Scope: uploaded batch file and embedded text/code sections reviewed in this research

VERDICT

The source is a high impact activation modification script. The reviewed code deliberately alters Windows/Office licensing state, replaces Office licensing related DLLs, modifies Software Protection Platform data, uses third-party KMS infrastructure, and can create hidden scheduled tasks running as SYSTEM. These behaviors present material integrity and operational risks. The source review does not, by itself, establish credential theft, personal-data exfiltration, or that the file is a general-purpose malware sample.

Executive Summary

This report consolidates the static review performed on the uploaded MAS_AIO.cmd batch file. The review covered startup and privilege handling, activation modes, Office DLL changes, TSforge licensing-store manipulation, KMS server selection and scheduled tasks, network behavior, defensive-evasion indicators, system repairs, cleanup, and limits of the evidence.

The file identifies itself as version 3.12. The previously reported SHA-256 for the collected script was 850F979665FB93999ACAE93F4790C1FF8ED2041532060B7966A121C2D29A0BFA; it reportedly matched the expected hash in its metadata. A match only establishes identity relative to the compared reference; it does not establish safety unless the reference is independently trusted.

The strongest source-supported findings are intentional licensing manipulation, custom Office licensing DLL replacement, direct modification of Software Protection Platform data, conditional high-privilege scheduled-task persistence, third-party KMS dependencies, and a potentially disruptive WMI repository repair path.

No script execution, live network capture, sandbox run, or independent reverse engineering of embedded DLLs was performed. This is a static source assessment, not a verdict about any particular computer.

Key Findings at a Glance

Finding	Potential impact	Confidence in source finding
Windows and Office activation state manipulation	Changes licensing behavior and registry/store state	High
Custom Office licensing DLLs and symbolic links	Software-integrity and compatibility risk	High
TSforge store parsing and rewriting	Licensing state may become inconsistent if interrupted or faulty	High
Hidden scheduled tasks running as SYSTEM	Conditional persistence and privileged recurring execution	High
Public third-party KMS server selection	External trust and availability dependency	High
WMI repair path can disable Winmgmt and delete repository	Potential disruption to Windows management functionality	High
PE timestamp modification and altered Base64 encoding	Provenance/inspection concern; encoding stated to reduce AV detection	High
Credential theft / unrelated personal-data exfiltration	Not established by reviewed excerpts	Undetermined

1. Scope, Methodology, and Limitations

The assessment was based on the uploaded file MAS_AIO_20261009_074136.cmd (19,228 lines, version 3.12) and source excerpts inspected during the conversation. Findings distinguish observed source behavior from inferred impact and unresolved questions.

Methodology

- Reviewed startup, argument dispatch, privilege elevation, and environment checks.
- Traced Windows activation, Office Ohook, TSforge, and KMS-related code paths.
- Inspected task creation, task XML, task payload extraction, and renewal logic.
- Reviewed network-related code, including Microsoft activation/diagnostic endpoints and public KMS server selection.
- Searched for selected defensive-evasion strings, including common Defender preference commands, encoded PowerShell invocation, and Invoke-Expression.
- Inspected selected repair and cleanup routines for service, registry, DLL, and WMI changes.

Limitations

- Static review cannot confirm which conditional branches execute on a particular host.
- Embedded sppc32.dll and sppc64.dll payloads were not independently extracted and reverse engineered as binaries in this review.
- No dynamic sandbox run, system-state comparison, packet capture, or endpoint telemetry was available.

- A string search with no matches does not prove equivalent behavior is absent under other names or APIs.
- A hash match only establishes identity relative to the compared reference; it does not prove safety.

2. File Identity and Startup / Dispatch Behavior

The script identifies itself as version 3.12 and sets up its execution environment, including PATH related variables and the system directory. It handles architecture-specific relaunch behavior, checks Windows build and PowerShell capabilities, detects whether it is running elevated, and can relaunch through PowerShell with the RunAs verb.

It parses command-line arguments for unattended operation and activation-mode selection. The reviewed menu exposes HWID activation, Ohook Office activation, TSforge, Online KMS, activation-status checks, Windows edition changes, Office edition changes, troubleshooting, and extras. Argument-based execution can select particular routines without navigating the interactive menu.

Startup logic also checks service state, Windows build compatibility, PowerShell language mode, .NET availability, and other environmental conditions. It performs update-related domain checks and may offer to open relevant web pages.

Source evidence: Startup and dispatch were reviewed in approximately lines 1–559; the uploaded script is the primary evidence source.

3. HWID Activation and Windows Licensing Changes

The HWID path creates or uses GenuineTicket.xml under the ClipSVC GenuineTicket directory, copies the generated ticket into place, restarts ClipSVC, and invokes clipup. The source also contains service-startup changes for licensing and related services, including ClipSVC, sppsvc, SLsvc, KeyIso, LicenseManager, and Winmgmt.

The script resets SkipRearm to 0 and changes selected Software Protection Platform registry values. It also contains a troubleshooting branch that removes selected licensing-related state and reinstalls system licenses. These operations can affect licensing behavior and system configuration; the source does not show that every change is applied during every launch.

Source evidence: Previously reviewed ranges include approximately lines 908–950 (GenuineTicket handling), 1680–1710 (service configuration), 2130–2168 (SkipRearm and SvcRestartTask cleanup), and 16390–16479 (SPP repair and licensing cleanup).

4. Office Ohook Activation and DLL Replacement

The Ohook routine detects installed Office versions, architecture, product IDs, installation paths, licensing files, and whether the installation uses Click-to-Run or MSI. It can invoke Office integrator/licensing functionality and adjust selected ProductReleaseIds and KMS-related values.

The install path removes or moves selected sppc*.dll / sppcs.dll files, creates symbolic links, and extracts custom DLL payloads into Office licensing directories. Other paths rename existing Office licensing DLLs and attempt to restore expected file names. The uninstall path also removes selected licensing-resiliency registry keys across profiles.

The source documents two embedded payloads, sppc32.dll and sppc64.dll, and includes code to alter PE timestamp fields and recalculate the PE checksum. Its comments explicitly state that altered encoding characters are used to reduce antivirus detection of the Base64 representation. This is a material security concern because file provenance and independent binary validation become especially important.

Payload	SHA-256 stated in script comments
sppc32.dll	09865ea5993215965e8f27a74b8a41d15fd0f60f5f404cb7a8b3c7757acdab02
sppc64.dll	393a1fa26deb3663854e41f2b687c188a9eacd87b23f17ea09422c4715cb5a9f

Source evidence: Relevant ranges include approximately lines 2730–2992 (Office detection and registry handling), 3190–3315 (DLL replacement/symlinks), and 4040–4130 (PE timestamp/checksum routine and embedded DLL documentation).

Assessment: custom DLL installation into licensing paths is confirmed. The payloads' full internal behavior is not established by reading the batch file alone.

5. TSforge and Software Protection Platform Store Manipulation

TSforge exposes Windows, ESU, Office, and advanced options, with methods such as StaticCID, ZeroCID, and KMS4k and selection logic that can depend on build/network conditions. The routine reads its own script content, splits at an internal marker, and creates PowerShell script blocks from extracted code.

The embedded C# helper contains logic to contact Microsoft's Batch Activation SOAP endpoint with activation-related data in applicable branches. It also contains byte arrays and helpers used to parse and modify licensing-store structures.

The reviewed store-manipulation code opens physical licensing store files for read/write, parses/decrypts their contents, changes selected blocks, serializes/encrypts the result, and writes data back. Related functions reset grace-period data, remove key change lock blocks, remove selected tamper flags, replace product key related blocks, and can delete cache.dat. A load path stops the Software Protection service, writes store data, and restarts the service.

These are consequential modifications to Windows licensing state. If a write fails, the process is interrupted, or the data is incompatible, licensing state could be inconsistent. This is a risk inference; the source review did not test whether any machine was damaged.

Source evidence: Relevant ranges include approximately lines 4272–4350 (TSforge modes), 5830–5912 (embedded code extraction and method description), 6140–6285 (activation SOAP helper), 7835–7890 (store load/write path), 8440–8540 (product-key/grace-period/key-change blocks), 8820–8865 (tamper flags), 9185–9249 (activation-related data), and 10159–10560 (store parsing/writing).

6. KMS Activation, Server Selection, and Registry Changes

The KMS path queries Windows and Office volume licensing products through WMI or PowerShell, attempts activation, and writes or clears KMS server/port values under licensing related registry paths. It uses the standard KMS port value of 1688 in the reviewed logic and can select from a list of public KMS hostnames. It resolves IPv4 addresses and includes a fallback address of 222.184.9.98.

The source includes NoGenTicket=1 and, on one Windows build, NoAcquireGT=1 policy values, which its comments describe as opting out of sending KMS client activation data to Microsoft. It also cleans up selected IFEO entries and KMS-related registry values.

Use of public third-party KMS servers creates a trust dependency on external infrastructure. This is distinct from Microsoft-documented enterprise KMS, which is intended to operate through an appropriately licensed and configured KMS host.

Source evidence: Relevant ranges include approximately lines 12620–12709 (KMS activation/task decision logic), 13348–13430 (activation task flow), 13433–13545 (registry cleanup and server selection), and 13510–13545 (public hostnames and fallback IP).

7. Scheduled Tasks and Conditional Persistence

The KMS renewal routine can create Activation-Renewal and Activation-Run_Once scheduled tasks. The XML configures the renewal task for a weekly Sunday trigger and the one time task for a user logon trigger. Both are configured to run as SYSTEM, at the highest run level, with hidden=true and a network-availability condition. The action launches Activation_task.cmd with a Task argument.

The script creates a temporary directory with a generated identifier, extracts task XML from its own contents, writes Activation_task.cmd by splitting its own batch file at a marker, writes Info.txt, and checks for expected task/file state. If setup checks fail, it attempts cleanup. The renewal task logs activity, checks network reachability, selects a KMS server, queries volume products, attempts activation, and may delete the one-time task in a relevant branch.

This is conditional persistence for activation renewal. High privilege and hidden configuration increase impact, but do not alone prove general purpose malware behavior. The source describes the one time task as self deleting, but this should not be treated as guaranteed across all branches or failures.

Source evidence: Relevant ranges include approximately lines 12970–13005 (cleanup), 13075–13200 (renewal payload behavior), 13490–13575 (task creation and payload extraction), and 13636–13757 (task XML).

8. Network Behavior and Privacy Considerations

The source performs connectivity diagnostics against Microsoft licensing/account/purchase endpoints, uses an HTTP SOAP request to the Microsoft Batch Activation endpoint in the TSforge helper, and retrieves Office Click-to-Run release/build metadata from Microsoft's Office device-management API. The KMS renewal logic uses ping and DNS tests to determine connectivity and resolve selected KMS servers.

These requests are broadly consistent with licensing, activation, diagnostics, and update-metadata workflows. However, the exact data sent depends on the execution branch and request construction. No network capture was performed.

The reviewed excerpts do not establish theft of passwords, browser data, personal documents, or unrelated user files. It would be inaccurate to claim data exfiltration has been proven. It would also be too strong to claim every possible data exposure has been ruled out without tracing all network-capable code paths and inspecting embedded binaries.

Source evidence: Relevant ranges include approximately lines 1000–1045 (Microsoft connectivity/diagnostic requests), 6140–6285 (activation SOAP helper), 13075–13200 (renewal network tests), 13510–13545 (KMS server selection), and 19120–19128 (Office release-data GET).

9. Defensive-Evasion Indicators and Negative Searches

Text searches were conducted for Add-MpPreference, Set-MpPreference, DisableRealtimeMonitoring, DisableAntiSpyware, WinDefend, ExclusionPath, EncodedCommand, FromBase64String, and Invoke Expression. The named Defender disabling and exclusion strings, EncodedCommand, and Invoke-Expression did not appear in those searches. FromBase64String did appear in code that decodes embedded license configuration data and related values.

A negative text search is not proof of absence. Equivalent behavior can be implemented through different commands, APIs, registry changes, or embedded binaries. The script contains an explicit comment that the encoded DLL representation was altered to reduce antivirus detection; this is a source-supported concern independent of whether common Defender commands are present.

The script also checks hosts-file entries for domains associated with antivirus vendors and checks for artifacts associated with third party activation utilities. These checks may be diagnostic; they do not, by themselves, prove that antivirus is disabled or that a host is infected.

Source evidence: Relevant ranges include approximately lines 1510–1569 (hosts-file and activator-artifact checks) and 4090–4130 (embedded payload and anti-detection comment).

10. Troubleshooting, Cleanup, and Reversibility

The script contains repair paths that stop and restart licensing services, delete selected SPP registry values, remove IFEO entries, reinstall licenses, and clean up KMS tasks and files. The WMI troubleshooting path may stop Winmgmt, attempt repository salvage, disable the service, and delete the WMI repository if branch conditions lead there.

Cleanup is not equivalent to a full rollback. The source does not consistently show that original registry values and service startup types are backed up and restored exactly. Restoring Office DLLs may depend on whether original files remain available and whether file operations succeeded. Removing tasks does not reverse all registry, service, licensing-store, or DLL changes.

Recovery recommendations if the script was run

1. Preserve the original script and record its hash; do not delete evidence before reviewing the system.
2. Use trusted installation media or known-good sources to validate Windows and Office licensing components.
3. Inspect scheduled tasks, especially Activation-Renewal and Activation-Run_Once, and check for leftover files under Program Files\Activation-Renewal.
4. Review KMS server and port registry values and relevant Software Protection Platform entries.
5. Verify startup configuration and operational state of services changed by the script.
6. If WMI repair was invoked, test WMI functionality and assess whether repository recovery is needed.
7. For high-assurance recovery after privileged execution, consider restoring a known-good backup or reinstalling Windows and Office from trusted media.

Source evidence: Relevant ranges include approximately lines 13433–13510 (KMS cleanup), 16390–16479 (SPP cleanup/repair), and 16720–16789 (WMI repair).

11. Risk Register

ID	Finding	Severity / impact	Confidence	Evidence basis
R1	Office licensing DLL replacement and symbolic links	High: integrity / compatibility	High	Ohook install/uninstall paths; embedded payloads
R2	TSforge modifies licensing-store blocks	High: licensing-state consistency	High	Embedded C# parsing and store write routines
R3	Hidden SYSTEM scheduled tasks	High: privileged conditional persistence	High	Task XML and renewal routine
R4	Public third-party KMS hosts	High trust concern	High	Host list, DNS resolution, registry configuration
R5	WMI repository deletion branch	High operational risk	High	Winmgmt disable/delete path
R6	PE timestamp edits / anti-detection encoding comment	Elevated forensic concern	High	PE modification routine and payload comments
R7	Registry/service/license changes	Moderate-high	High	Activation and troubleshooting paths
R8	Credential theft / unrelated data exfiltration	Undetermined	Not established	No direct proof in reviewed excerpts; analysis incomplete

12. Overall Assessment

The evidence supports classifying MAS_AIO.cmd as a high-impact activation-modification script with material software integrity and system configuration risks. The file is intentionally designed to alter Windows and Office licensing behavior; it includes Office DLL replacement, direct licensing store modification, thirdparty KMS dependencies, and conditional scheduled tasks running as SYSTEM.

The evidence reviewed does not support a categorical claim that the file is a credential stealer or that every launch infects a computer. It also does not justify declaring the file safe. A definitive malware assessment requires independent binary analysis and, where authorized, controlled execution in an isolated test environment.

Recommended disposition

Do not run this script on a primary or sensitive computer if system integrity and security are priorities. If it has already run with administrator privileges, treat the host as having undergone significant licensing and configuration changes until verified. Prefer official licensing and activation methods for systems that must remain trusted.

13. Follow-up Work Required for a Higher-Confidence Verdict

- Extract both embedded DLLs in an isolated analysis environment and validate the documented SHA-256 values.
- Inspect DLL imports, exports, strings, metadata, signatures, and behavior; compare with trusted source builds if available.
- Perform control flow analysis of all code that creates processes, writes files, changes registry values, or sends network requests.
- Use a disposable VM snapshot to compare file, registry, service, scheduled-task, and licensing-store state before and after each activation path.
- Capture DNS and network traffic during controlled tests and inspect request bodies, destinations, and response handling.
- Verify the provenance of the expected script hash and the embedded DLL hashes.
- Document cleanup success and remaining state after each routine without assuming uninstall paths fully reverse changes.

14. External Context (Separate from Source Findings)

Official Microsoft documentation was consulted only to contextualize standard enterprise KMS behavior. It does not independently validate or endorse the script under review.

- Microsoft Learn, “Overview of volume activation of Office”: KMS is a client-server volume activation model; MAK and Active Directory based activation are alternative methods. <https://learn.microsoft.com/en-us/deployoffice/vlactivation/plan-volume-activation-of-office>
- Microsoft Learn, “Activate volume licensed versions of Office by using KMS”: describes KMS client configuration, caching, and default port 1688. <https://learn.microsoft.com/en-us/office/volume-license-activation/activate-office-by-using-kms>
- Microsoft Learn, “Key Management Services (KMS) activation planning for Windows Server”: describes KMS as volume activation using a KMS host and outlines operational requirements. <https://learn.microsoft.com/en-us/windows-server/get-started/kms-activation-planning>

Context distinction: Microsoft documentation describes supported volume activation workflows. It should not be interpreted as approval of public third-party KMS servers, custom Office DLL hooks, or licensing-store modifications found in this script.

Appendix A. Source Line-Range Index

Topic	Reviewed source ranges (approximate)
Startup, arguments, elevation, environment checks	1–559
HWID GenuineTicket and activation workflow	908–950; 1680–1710
Microsoft connectivity diagnostics	1000–1045
Hosts-file / third-party activator checks	1510–1569
SkipRearm and service-task cleanup	2130–2168
Office Ohook detection and licensing operations	2730–2992
Office DLL replacement and symlinks	3190–3315
PE timestamp/checksum editing and DLL payload notes	4040–4130
TSforge methods and embedded code	4272–4350; 5830–5912
TSforge activation helper / SOAP request	6140–6285
TSforge store load/write and block manipulation	7835–7890; 8440–8540; 8820–8865; 10159–10560
KMS activation and task decision logic	12620–12709
KMS renewal payload	13075–13200
KMS registry cleanup and server selection	13433–13545
Task XML definitions	13636–13757
SPP repair and licensing cleanup	16390–16479
WMI repair path	16720–16789
Office release-data HTTP GET	19120–19128

Appendix B. Evidence Language and Confidence

“Observed” means the behavior is visible in the inspected source excerpt. “Potential impact” means a plausible consequence inferred from that behavior, not an observed result on a live system. “Not established” means the reviewed evidence does not support the claim; it does not prove the claim is impossible. “High confidence” refers to confidence that the code is present in the inspected source, not confidence that it executes on every machine.

This report is a technical research summary, not legal advice, a software authenticity certificate, or a forensic determination of compromise.